

CS236

LabQuiz2 Grading Rubric

Q1

Marks are given for the testcases given in the Lab exam. 0 marks are given for a testcase if it doesn't compile.

- 1) p1a 8 - 1.5 marks
- 2) p1a 9 - 1.5 marks
- 3) p1b - 2 marks
- 4) p1c - 2 marks
- 5) p1d - 1 marks

Q2

Five test cases were used to verify the correct implementation. If the test case consistently runs correctly, full marks for the test case are to be awarded. If the test case fails, or passes due to hardcoding, or only passes some of the time, no marks are to be given for the test case.

Test Case 1 (4 marks)

Tested via p2a and p2b.

Basic functionality of `fork()+exec()`: upon being passed a valid path and arguments, `spawn()` correctly creates a child process, loads the correct image, copies file descriptors, passes the arguments to the child process and returns its PID. The PID of the child process should be the next available PID, without skipping any in the middle, and no process slots should be left in a USED state.

Test Case 2 (1.5 marks)

Tested via p2c.

Alternative functionality of just forking: upon being passed (0, 0) as arguments, `spawn()` should behave exactly like `fork()`.

Test Case 3 (1 mark)

Tested via internal test case:

C/C++

```
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[])
{
    char *args[] = {"/invalid/path", 0};
    int ret = spawn("/invalid/path", args);
    if (ret < 0)
        printf("[pid %d] Invalid path successfully failed\n", getpid());
    else
        printf("[pid %d] Incorrect behaviour\n", getpid());

    exit(0);
}
```

Proper error handling: Upon being passed a non-existent path, `spawn()` should fail and return a negative value. If a new process is created afterwards, skipping one PID is allowed. No process slots should be left in a USED state.

Test Case 4 (1.5 marks)

Tested via internal test case:

C/C++

```
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[])
{
    char *args1[] = {"cat", "README", 0};
    printf("[pid %d] Spawning cat\n", getpid());
    int pid = spawn("cat", args1);
    if (pid <= 0)
        exit(pid);
    int wpid = 0;
    wpid = wait(0);
    if (wpid != pid)
        exit(wpid);
    printf("[pid %d] cat exited\n", getpid());
    char *args2[] = {"echo", "Spawn", 0};
```

```

printf("[pid %d] Spawning echo\n", getpid());
pid = spawn("echo", args2);
if (pid <= 0)
    exit(pid);
wpid = wait(0);
if (wpid != pid)
    exit(wpid);
printf("[pid %d] echo exited\n", getpid());

exit(0);
}

```

Multiple calls: `spawn()` should behave in the same manner when called multiple times, with different, but valid, paths and arguments.

Q3

The four cases given below correspond to the testcases p3a, p3b, p3c, p3d given in the question, each carrying 2 marks. 0 marks are given for a testcase if it doesn't compile.

1. Virtual address `FASTUPTIME_PAGE` should be readable and not writable from userspace - 2 marks
2. Virtual address `FASTUPTIME_PAGE` should be mapped to the same physical address in all userspace processes - 2 marks
3. `uptime()`, `fastuptime()` must be close both at start & end, we kept a threshold of 3 ticks for this. `#ticks for fastuptime() <= #ticks of uptime()` - 2 marks
4. They should work correctly for multiple concurrent processes: `uptime()`, `fastuptime()` should be close (threshold is 3 ticks) in all the processes (parent, all children) - 2 marks